RFID Runtime Playbook: 11 Examples (Kotlin Edition)

This document maps runtime log patterns to the exact code paths in app/src/main/java/com/zebra/rfid/demo/sdksample/MainActivity.kt and app/src/main/java/com/zebra/rfid/demo/sdksample/RFIDHandler.kt.

How to use this file: - Match your log sequence to one example. - Verify source anchors first. - Compare expected status strings. - Use thread context to debug timing issues.

Eleven Examples

1. Example One: Init and Connect
2. Example Two: Background and Disconnect
3. Example Three: Foreground and Reconnect
4. Example Four: Battery Remove and Disconnect
5. Example Five: Battery Insert and Reconnect
6. Example Six: RFD40 Sled Detach and Disconnect
7. Example Seven: RFD40 Sled Attach and Reconnect
8. Example Eight: USB Cable Connected -> Reader Disappear and Disconnect
9. Example Nine: USB Cable Unplug and Reconnect Window
10. Example Ten: Power Connected -> Disconnected -> Reconnect
11. Example Eleven: Power Disconnected While Already Connected
12. Example Twelve: Verified Power-Plug Disconnect and Power-Unplug Reconnect Trace

1) Example One: Init and Connect

Thread context

- Main thread: activity lifecycle (`onCreate`, `onPostResume`)
- Background executor: SDK init (`createInstanceAndConnect`) and connection task
- Scheduled executor: deferred connect (`finishConnectionAttempt(connect())`)

Source anchors

- `MainActivity.kt`: `initializeRfidHandlerIfPermitted`
- `MainActivity.kt`: `onPostResume`
- `RFIDHandler.kt`: `onCreate`
- `RFIDHandler.kt`: `onResume`
- `RFIDHandler.kt`: `initSdk`
- `RFIDHandler.kt`: `connectReader`
- `RFIDHandler.kt`: `selectReader`
- `RFIDHandler.kt`: `connect`

Expected status strings

- `Initializing reader...` or `Preparing reader...`
- `Connecting...`
- `Connected: <reader-hostname> (<ms> ms)`

Representative snippet

```
// MainActivity -> RFIDHandler bootstrap
private fun initializeRfidHandlerIfPermitted() {
    if (!canUseRfidHandler() || rfidHandlerInitialized) return
    Log.d(TAG, "Step 2: rfidHandler.onCreate for this MainActivity with UI Response Interface")
    rfidHandler!!.onCreate(this, this)
    rfidHandlerInitialized = true
}

override fun onPostResume() {
    super.onPostResume()
    Log.d(TAG, "STEP: onPostResume requestReaderResumeDebounced")
    requestReaderResumeDebounced("activity_post_resume", false)
}

// RFIDHandler connect pipeline
private fun initSdk() {
    Log.d(TAG, "STEP: InitSDK")
    if (readers == null) {
        if (initializationInProgress) {
            Log.d(TAG, "STEP: Skip Duplicated InitSDK")
            return
        }
        Log.d(TAG, "STEP: InitSDK initializationInProgress, createInstanceAndConnect")
        initializationInProgress = true
        runOnBackground { createInstanceAndConnect() }
    } else if (resumeRequested) {
        connectReader()
    }
}
```

2) Example Two: Background and Disconnect

Thread context

- Main thread: activity pause callback
- Background executor: beep tone operation

Source anchors

- `MainActivity.kt`: `onPause`
- `RFIDHandler.kt`: `onPause`
- `RFIDHandler.kt`: `disconnect`

Expected status strings

- Detach path: `Reader detached: <name>`
- Manual/background path: `Disconnected`

Representative snippet

```
// MainActivity
override fun onPause() {
    super.onPause()
    stopPowerReconnectWindow()
    if (rfidHandlerInitialized) {
        Log.d(TAG, "STEP: onPause, disconnect")
        rfidHandler!!.onPause()
    }
}

// RFIDHandler
fun onPause() {
    Log.d(TAG, "STEP: RFIDHandler onPause: disconnecting")
    resumeRequested = false
    connectionInProgress = false
    disconnect()
}
```

3) Example Three: Foreground and Reconnect

Thread context

- Main thread: resume debounce and status update
- Background executor: connection task inside `connectReader`

Source anchors

- `MainActivity.kt`: `requestReaderResumeDebounced`
- `MainActivity.kt`: `requestReaderResume`
- `RFIDHandler.kt`: `onResume`

Expected status strings

- `Connecting...`
- `Connected: <reader-hostname> (<ms> ms)`

Representative snippet

```
// MainActivity
private fun requestReaderResumeDebounced(source: String, force: Boolean) {
    val now = SystemClock.elapsedRealtime()
    if (!force && now - lastReaderResumeRequestAtMs < READER_RESUME_DEBOUNCE_MS) {
        Log.d(TAGUSB, "Skipping reconnect request due to debounce window. source=$source")
        return
    }
    lastReaderResumeRequestAtMs = now
    requestReaderResume()
}
```

```
private fun requestReaderResume() {
    if (!rfidHandlerInitialized) {
        initializeRfidHandlerIfPermitted()
    }
    if (!rfidHandlerInitialized) {
        applyReaderStatus(getString(R.string.status_permission_required))
        return
    }
    val result = rfidHandler!!.onResume()
    if (result.isNotEmpty() && !shouldSuppressReconnectStatus(result)) {
        applyReaderStatus(result)
    }
}
```

4) Example Four: Battery Remove and Disconnect

Thread context

- SDK callback thread: `RFIDReaderDisappeared`
- Background executor: beep tone

Source anchors

- `RFIDHandler.kt`: `RFIDReaderDisappeared`
- `RFIDHandler.kt`: `disconnect`

Expected status strings

- `Reader detached: <name>`
- `Disconnected` may be suppressed for detach events

Representative snippet

```
override fun RFIDReaderDisappeared(readerDevice: ReaderDevice) {
    Log.d(TAG, "RFIDReaderDisappeared " + readerDevice.name)
    detachedEventInProgress = true
    responseHandler?.sendToast("Reader detached: " + readerDevice.name)
    disconnect()
}
```

5) Example Five: Battery Insert and Reconnect

Thread context

- SDK callback thread: `RFIDReaderAppeared`
- Scheduled executor: deferred `connectReader`

Source anchors

- `RFIDHandler.kt`: `RFIDReaderAppeared`

- RFIDHandler.kt: beepAppear
- RFIDHandler.kt: connectReader

Expected status strings

- Reader attached: <name>
- Connecting...
- Connected: <reader-hostname> (<ms> ms)

Representative snippet

```
override fun RFIDReaderAppeared(readerDevice: ReaderDevice) {
    Log.d(TAG, "RFIDReaderAppeared " + readerDevice.name)
    responseHandler?.sendToast("Reader attached: " + readerDevice.name)
    beepAppear()
    scheduleOnBackground({ connectReader() }, READER_APPEAR_DELAY_MS)
}
```

6) Example Six: RFD40 Sled Detach and Disconnect

Thread context

- Same as Example Four (shared callback path)

Source anchors

- RFIDHandler.kt: RFIDReaderDisappeared
- RFIDHandler.kt: disconnect

Expected status strings

- Reader detached: <name>

Representative snippet

```
// Shared with Example Four
override fun RFIDReaderDisappeared(readerDevice: ReaderDevice) {
    Log.d(TAG, "RFIDReaderDisappeared " + readerDevice.name)
    detachedEventInProgress = true
    responseHandler?.sendToast("Reader detached: " + readerDevice.name)
    disconnect()
}
```

7) Example Seven: RFD40 Sled Attach and Reconnect

Thread context

- Same as Example Five (shared callback path)

Source anchors

- `RFIDHandler.kt`: `RFIDReaderAppeared`
- `RFIDHandler.kt`: `connectReader`

Expected status strings

- `Reader attached: <name>`
- `Connected: <reader-hostname> (<ms> ms)`

Representative snippet

```
// Shared with Example Five
override fun RFIDReaderAppeared(readerDevice: ReaderDevice) {
    Log.d(TAG, "RFIDReaderAppeared " + readerDevice.name)
    responseHandler?.sendToast("Reader attached: " + readerDevice.name)
    beepAppear()
    scheduleOnBackground({ connectReader() }, READER_APPEAR_DELAY_MS)
}
```

8) Example Eight: USB Cable Connected -> Reader Disappear and Disconnect

Thread context

- Main thread: USB broadcast (`ACTION_POWER_CONNECTED` log and USB-sharing modes check)
- SDK callback thread: detach event

Source anchors

- `MainActivity.kt`: `mBatteryReceiver`
- `RFIDHandler.kt`: `RFIDReaderDisappeared`

Expected status strings

- USB log: `USB Client action: android.intent.action.ACTION_POWER_CONNECTED`
- If power-only mode: `RFID disconnected while USB power cable connected` or `Disconnected`
- If USB file transfer: `USB file transfer active or debug mode\r\nRFID Disconnected\r\nWait for USB cable unplug for reconnect`

Representative snippet

```
if (Intent.ACTION_POWER_CONNECTED == action) {
    refreshUsbModeFromStickyState()
    if (powerConnectedLatched) {
        Log.d(TAGUSB, "STEP: Skip ACTION_POWER_CONNECTED because latch is active (waiting for c
        return
    }
    val now = SystemClock.elapsedRealtime()
    if (now - lastPowerConnectedHandledAtMs < POWER_CONNECTED_EVENT_DEBOUNCE_MS) {
        Log.d(TAGUSB, "STEP: Skip duplicate ACTION_POWER_CONNECTED within debounce window")
```

```
            return
        }
        lastPowerConnectedHandledAtMs = now
        powerConnectedLatched = true

        Log.d(TAGUSB, "STEP: ACTION_POWER_CONNECTED")
        Log.d(TAGUSB, "ACTION_POWER_CONNECTED usbState: connected=$usbConnected, configured=$usbCo
        val handlerBusy = rfidHandlerInitialized && rfidHandler != null && rfidHandler!!.isConnecti
        Log.d(TAGUSB, "ACTION_POWER_CONNECTED state: reconnectWindowActive=$powerReconnectWindowAct

        if (powerReconnectWindowActive) {
            Log.d(TAGUSB, "STEP: ACTION_POWER_CONNECTED during reconnect window -> cancel reconnect
        }
        stopPowerReconnectWindow()
        suppressReconnectStatusUntilConnected = false

        if (!rfidHandlerInitialized || rfidHandler == null) return

        if (rfidHandler!!.isTC22R()) {
            Log.d(TAGUSB, "STEP: TC22R, ignore ACTION_POWER_CONNECTED")
            onReaderStatusUpdate("Connected to TC22R")
            return
        }

        if (usbFileTransferModeActive) {
            rfidHandler!!.setSkipTc53eReaderSelection(false)
            Log.d(TAGUSB, "STEP: ACTION_POWER_CONNECTED in file-transfer mode -> keep RFID connecte
            sendToast("USB file transfer active or debug mode\r\nRFID Disconnected\r\nWait for USB
            return
        }

        rfidHandler!!.setSkipTc53eReaderSelection(true)
        Log.d(TAGUSB, "STEP: ACTION_POWER_CONNECTED power-only -> disconnect RFID")
        rfidHandler!!.onPause()
        sendToast("RFID disconnected while USB power cable connected")
    }
```

9) Example Nine: USB Cable Unplug and Reconnect Window

Thread context

- Main thread: USB broadcast + reconnect window orchestration
- Main handler scheduled tasks: retry attempts and timeout
- Background executor: RFID connection work in handler

Source anchors

- `MainActivity.kt`: `mBatteryReceiver`
- `MainActivity.kt`: `startPowerReconnectWindow`

- `MainActivity.kt: powerReconnectAttemptRunnable`
- `MainActivity.kt: shouldSuppressReconnectStatus`

Expected status strings

- `Starting power-unplug reconnect window`
- `Power reconnect attempt <n>/3`
- `Power reconnect deferred: handler busy`
- transient suppression logs until connected

Failure variant (important)

- First or early attempt may return `Connection failed: RFID_COMM_OPEN_ERROR`.
- This is expected during cable transition races and should recover via bounded retries.

Representative snippet

```
if (Intent.ACTION_POWER_DISCONNECTED == action) {
    Log.d(TAGUSB, "ACTION_POWER_DISCONNECTED")
    powerConnectedLatched = false
    usbConnected = false
    usbConfigured = false
    usbFileTransferModeActive = false
    if (rfidHandler == null) return
    rfidHandler!!.setSkipTc53eReaderSelection(false)
    if (rfidHandler!!.isReaderConnected()) {
        stopPowerReconnectWindow()
        Log.d(TAG, "Reader already connected")
        sendToast("USB unplugged. Reader Connected")
        return
    }
    startPowerReconnectWindow()
}
```

10) Example Ten: Power Connected -> Disconnected -> Reconnect

Thread context

- Main thread: power event sequence and reconnect suppression
- SDK callback thread: detach/attach callbacks during cable transitions
- Main handler scheduled tasks: retry loop while busy

Source anchors

- `MainActivity.kt: mBatteryReceiver`
- `MainActivity.kt: startPowerReconnectWindow`
- `MainActivity.kt: powerReconnectAttemptRunnable`
- `RFIDHandler.kt: connectReader`

Expected status strings

- `USB Client action: android.intent.action.ACTION_POWER_CONNECTED`
- `ACTION_POWER_DISCONNECTED`
- `Power reconnect deferred: handler busy`
- final `Connected: ...` status

Failure variant (important)

- During rapid cable transitions, reader selection can temporarily pick host reader (`DEFAULT`) before external reader reappears.
- This is acceptable if final status reaches connected and suppression clears.

Representative snippet

```
private val powerReconnectAttemptRunnable: Runnable = object : Runnable {
    override fun run() {
        if (!powerReconnectWindowActive) return
        if (rfidHandler == null || (rfidHandlerInitialized && rfidHandler!!.isReaderConnected()
            stopPowerReconnectWindow()
            return
        }
        if (rfidHandlerInitialized && rfidHandler!!.isConnectionBusy()) {
            Log.d(TAGUSB, "Power reconnect deferred: handler busy")
            mainHandler.postDelayed(this, POWER_RECONNECT_BUSY_RETRY_DELAY_MS)
            return
        }
        val attemptNumber = powerReconnectAttemptIndex + 1
        Log.d(TAGUSB, "Power reconnect attempt $attemptNumber/${POWER_RECONNECT_ATTEMPT_DELAYS_
        requestReaderResumeDebounced("power_disconnected_retry_$attemptNumber", true)
        powerReconnectAttemptIndex++
        if (powerReconnectAttemptIndex < POWER_RECONNECT_ATTEMPT_DELAYS_MS.size) {
            mainHandler.postDelayed(this, POWER_RECONNECT_ATTEMPT_DELAYS_MS[powerReconnectAtte
        }
    }
}
```

11) Example Eleven: Power Disconnected While Already Connected

Thread context

- Main thread only: receiver short-circuit path

Source anchors

- `MainActivity.kt`: `mBatteryReceiver`

Expected status strings

- `ACTION_POWER_DISCONNECTED`

- Reader already connected
- USB unplugged. Reader Connected

Representative snippet

```
if (Intent.ACTION_POWER_DISCONNECTED == action) {
    Log.d(TAGUSB, "ACTION_POWER_DISCONNECTED")
    powerConnectedLatched = false
    usbConnected = false
    usbConfigured = false
    usbFileTransferModeActive = false
    if (rfidHandler == null) return
    rfidHandler!!.setSkipTc53eReaderSelection(false)
    if (rfidHandler!!.isReaderConnected()) {
        stopPowerReconnectWindow()
        Log.d(TAG, "Reader already connected")
        sendToast("USB unplugged. Reader Connected")
        return
    }
    startPowerReconnectWindow()
}
```

12) Example Twelve: Verified Power-Plug Disconnect and Power-Unplug Reconnect Trace

Thread context

- `RFIDSerialIOMgr`: Low-level USB error during bus release.
- Main thread: Power broadcast events.
- SDK callback thread: Hardware detach/attach events.
- Background executor: RFID session restoration.

Log Signature (21:02 Trace)

```
21:02:02.719 RFIDSerialIOMgr E Run ending due to exception: java.io.IOException: Queueing USB
21:02:02.895 RFID_SAMPLE_USB D USB Client action: android.intent.action.ACTION_POWER_CONNECTED
21:02:02.898 RFID_SAMPLE     D RFIDReaderDisappeared RFD4030-G00B700-US::
21:02:03.676 RFID_SAMPLE_USB D USB Client action: android.intent.action.ACTION_POWER_DISCONNEC
21:02:03.676 RFID_SAMPLE_USB D Starting power-unplug reconnect window
21:02:04.177 RFID_SAMPLE_USB D Power reconnect attempt 1/3
21:02:04.716 RFID_SAMPLE     D Found RFID Readers Size = 1
21:02:04.717 RFID_SAMPLE     D Selected reader idx=0, transport=DEFAULT, reason=Defaulted to f
21:02:06.619 RFID_SAMPLE     D STEP: Reader Connected in 1900ms
```

Analysis

1. **Transport Error**: `RFIDSerialIOMgr` detects the USB bus change before the Android `ACTION_POWER_CONNECTED` broadcast arrives.
2. **Hardware Release**: The SDK fires `RFIDReaderDisappeared` as the internal USB hub switches from data mode to power-sharing mode.

3. **User Action**: Cable is unplugged 0.8s later (`ACTION_POWER_DISCONNECTED`).
4. **Recovery**: The app waits 0.5s, starts attempt 1, detects the re-enumerated hardware (`RFIDTC53E`), and restores the session in 1.9s.

Suggestions for Ongoing Maintenance

- Keep source anchors current after refactors.
- Keep expected status strings synchronized with `onReaderStatusUpdate` and toast text.
- Preserve failure variants for cable-transition examples; they reduce false-positive bug reports.
- If new logs are added, include thread context first, then snippets.